Puneetheswar Potela

(+91)9392229814 | potelapuneetheswar@gmail.com | LinkedIn | Github

Education

Indian Institute of Technology Madras — Chennai, Tamil Nadu
B.Tech in Electrical Engineering, Minor in Computing; CGPA: 7.36/10 — *2021 – 2025*

Relevant Coursework
Advanced Computer Architecture(G), Computer Architecture(G), CAD for VLSI(G), Digital Systems Testing(G), Embedded Memory Design(G), Computer Organization, Microprocessors
**G - Graduate level course*

Technical Skills

Programming Languages: C, C++, Python, Bash
Assembly: ARM, RISCV
HDL: Verilog, Systemverilog, Bluespec SV
Protocols: SPI, UART, I2C, CAN, AXI4-LITE, JTAG
Debugging/Build Tools: GDB, OpenOCD, Make, CMake, Git
Hardware/EDA Tools: Yosys, nextpnr, Iverilog, GTKwave, Verilator
Hardware Platforms: STM32F103RB Nucleo, Raspberry Pi 3A+, Tang Nano 9k
Systems/Concepts: RTOS, Bare-Metal Development, Memory Mapped IO/peripheral driver development

Experience

OLA Electric Technologies — June 2025 – April 2026
Assistant Manager(Electrical and Electronics Team) — *Bangalore, Karnataka*

Auxiliary Battery for OLA S1 Platform(EV):

- Engineered low-voltage **AUX battery module** to retain handle, seat lock access during **main-pack power loss**
- Optimized cell configurations and circuit behavior for **dynamic current loads** while adhering to thermal limits
- Maintained **supply chain consistency** by repurposing existing **main-pack cells** for the AUX battery design

OLA Roadster and Diamondhead Platforms(EV):

- Developed the **Electrical Systems Architecture** for the **OLA Roadster** and **Diamondhead EV platforms**
- Designed **signal routing**, **power distribution**, and **pinouts** for core ECUs like **BMS**, **MCU**, **LCM**, **VCU**
- Standardized the front and rear **junction boxes** via **common flat cable architecture** across EV variants
- Executed critical hardware bring-up, and validation to deliver the **Diamondhead announcement prototype**

OLA Shakti Platform(BESS):

- Spearheaded **electrical architecture**, defining **power topologies**, **isolation barriers**, and **components**
- Built and validated the compliance-ready **homologation prototype** to meet the regulatory safety standards
- Defined **End-of-Line(EOL)** test specifications, pass/fail thresholds for high-voltage and thermal safety limits
- Trained an **8-member team** at OLA Futurefactory in **assembling procedures and EOL test requirements**
- Resolved the **cross-functional integration dependencies** by bridging the **hardware** and the **firmware teams**

Mindgrove Technologies — May 2024 – July 2024
RTL Design Intern — *Chennai, Tamil Nadu*

Indirect Branch Predictor (RISC-V Shakti C-Class):

- Analyzed pipeline architecture to implement **dynamic branch target buffering** for optimized instruction fetch
- Engineered a **target address buffer** with a **two-stage cascaded predictor** to balance latency and accuracy
- Enhanced decode logic to seamlessly support **32-bit Base** and **16-bit RISC-V Compressed** instruction sets
- Integrated the validated predictor module into the production-grade **Shakti C-Class core** pipeline
- Validated performance via **CoreMark benchmarks**, achieving **99% accuracy** with zero pipeline regressions

Projects

Custom Preemptive RTOS | *C, ARM Assembly, Embedded Systems, Bare-Metal, GDB* | *Details*

RTOS Kernel:

- Developed a bare-metal, preemptive **Round-Robin** scheduler utilizing SYSTICK-based preemption to handle multi-tasking execution workflows on an **ARM Cortex-M3 (STM32F103RB)** microcontroller
- Designed and implemented a strict **Kernel-Port contract** to explicitly decouple core kernel mechanics from architecture specific port layers, enabling highly modular and **platform-independent** software execution
- Developed deterministic synchronization primitives, including blocking **Semaphores** and **Mutexes**, to systematically guarantee thread-safe shared resource contention and reliably prevent critical data races
- Designed an explicit **task lifecycle state tracking** system with support for **task deletion** and **self deletion**
- Designed kernel instrumentation hook to analyze **real-time timing performance** metrics via **GDB scripts**
- Observed $\mathcal{O}(1)$ **74-cycle context switch** and $\mathcal{O}(n)$ **65 cycles per task** tick overhead on ARM Cortex-M3

Bare-metal UART driver:

- Programmed a custom bare-metal, **interrupt-driven** UART driver utilizing **ring-buffered circular queues** to successfully enable continuous, non-blocking, and asynchronous serial communication with minimal CPU overhead
- Designed an interactive **UART-based CLI Shell** acting as the user interface to **demonstrate kernel features**

Custom RISC-V SOC | *Verilog, RISC-V ISA, AXI4-Lite, FPGA, Spike Simulator* | *Details*

Core and Modules:

- Designed and implemented synthesizable **RV32I** ISA based RISC-V **multi-cycle** processor core written in verilog
- Extended the core with **Zicsr** extension and supporting **Machine-mode** timer interrupt and external interrupts
- Engineered **non-vectored** interrupt mechanism that routes all traps, exceptions to a **centralized trap handler**
- Developed **automated verification** system ensuring 100% architectural compliance with **Spike ISA simulator**
- Integrated the processor core, memory controllers, and memory-mapped peripherals over an **AXI4-Lite** bus interconnect protocol to ensure modular System-on-Chip expansion with well-defined **address-mapped regions**
- Designed a **hardware timer peripheral** fully compliant with the RISC-V Privileged Specification, exposing standard **mtime** and **mcycle** registers to the software layer for timer interrupts and cycle count book-keeping
- Developed a custom GPIO peripheral module, mapped it to the AXI4-Lite bus, and successfully targeted, routed, and validated the complete system on chip design on a **Tang Nano 9K FPGA** utilizing physical onboard LEDs

HW-SW Co-Verification | *Hardware-Software Co-Design, SoC Porting, System Simulation* | *Details*

RTOS and RISC-V SoC Integration & Profiling:

- Ported the above custom **RTOS kernel** to the above custom **RISC-V SOC**, aligning software context-saving mechanism with the hardware's minimal **common trap handler** philosophy while conforming to the RISC-V ABI
- Engineered custom **simulation testbenches** optimized for printing granular **execution traces** and exporting VCD waveforms, reducing debug cycles during **hardware-software debugging** across RTL and firmware layers
- Designed specialized system-level **simulation harness** to measure the exact cycle-accurate **hardware/software timing interactions** of the preemptive kernel running on top of the RISC-V SOC to validate scheduling behavior
- Observed a **987 cycle** minimum trap overhead and **1013 cycle** average trap overhead on the kernel's RISC-V port